



Phage
SECURITY

SECURITY REVIEW · PREPARED FOR

Bounce Tech

A leveraged token protocol
on HyperEVM

DATE

15.04.2026

CONDUCTED BY

Pyro, Lead Security Researcher
BengalCatBalu, Lead Security Researcher
Kvar, Security Researcher
YanecaB, Security Researcher

COMMIT

691b365

REMEDIATION

a7f0ccb

01 ABOUT

Phage Security

Phage Security is a team of highly skilled smart-contract security researchers collaborating with 10+ proven freelancers who have excelled in public contests and private audits alike.

With numerous vulnerabilities uncovered and patched across our completed audits, we strive to deliver the absolute best security service and client experience. While 100% security can never be guaranteed, we are committed to giving our utmost effort to protect your protocol.

PREVIOUS WORK

github.com/phage-security

REACH OUT

Telegram · @Pyro3b

WHAT AN AUDIT IS

An audit is a time, resource, and expertise-bound effort in which trained researchers evaluate smart contracts using a combination of manual and automated techniques to identify as many vulnerabilities as possible.

An audit can reveal the presence of vulnerabilities, but it cannot guarantee their absence. This report is a point-in-time assessment of the code at the commits named on the cover. It is not a warranty that the system is secure.

02 EXECUTIVE SUMMARY

FINDINGS	FIXED & VERIFIED	ACKNOWLEDGED
5	1	4

SEVERITY DISTRIBUTION



CLIENT	REPOSITORY	METHODS
Bounce Tech	github.com/bounce-tech/bounce-contracts	Manual review

CONTRACTS IN SCOPE

https://github.com/bounce-tech/bounce-contracts/pull/192	https://github.com/bounce-tech/bounce-contracts/pull/193
https://github.com/bounce-tech/bounce-contracts/pull/194	https://github.com/bounce-tech/bounce-contracts/pull/195
https://github.com/bounce-tech/bounce-contracts/pull/196	https://github.com/bounce-tech/bounce-contracts/pull/199
https://github.com/bounce-tech/bounce-contracts/pull/200	https://github.com/bounce-tech/bounce-contracts/pull/201

TIMELINE · APRIL 2026

Audit · 2 days		Remediation · 1 day	
KICK-OFF	END OF AUDIT	REMEDICATIONS START	REMEDICATIONS END
08.04.2026	09.04.2026	15.04.2026	15.04.2026

STARTING COMMIT HASH	691b3659dc817f182d95702cc9ac468ec4719aaa
FINAL COMMIT HASH	a7f0ccb5653d821893351461400582cbc30b3c65

03 PROJECT SUMMARY

What Bounce Tech is

Bounce is a leveraged token protocol on HyperEVM where users mint and redeem ERC20 tokens representing leveraged long/short positions, with the actual leverage managed off chain via Hyperliquid Perpetual Futures.

Protocol fees flow to BOUNCE stakers, and an off chain automation layer bridges funds between the contracts and Hyperliquid to maintain target leverage and process queued redemptions. Each token accrues a streaming fee through a checkpoint on every user interaction and settles redemptions the contract cannot cover instantly through a prepare-and-execute queue driven by whitelisted executors.

SECURITY POSTURE

BEFORE REMEDIATION

691b365

Checkpoint fee accrual

M-01 · M-02 · L-02

`_checkpoint` integrates the single current `streamingFee` over the whole elapsed window and freezes `lastCheckpoint` whenever the period fee rounds to zero. Rate changes, zero-fee stretches, and mints after idle periods all repriced elapsed time under inputs that never applied to it.

Fee split rounding

L-01

The treasury split in `_payFees` rounds down, so small period fees routed entirely to the `feeHandler` while the checkpoint still advanced, leaking the treasury's share.

Global parameter granularity

L-03

`minTransactionSize` and `executeRedemptionFee` are single global values, so the owner could not loosen them for one depreciated LeveragedToken without weakening every healthy one, locking depreciated holders out of redemption.

AFTER REMEDIATION

a7f0ccb

One of five findings fixed and verified. The fix advances `lastCheckpoint` when `streamingFee` is zero, so a later rate restoration can no longer charge a retroactive streaming fee for the whole zero-fee window, while keeping the anti-rounding accumulation for short periods with a live rate.

The four acknowledged items stay open as accepted tradeoffs. A rate change still reprices un-checkpointed time until agents sweep every token around the change, small fee splits can still round the treasury share to zero, and the redemption parameters stay global rather than per token. None of them is a permissionless path to user funds, and each has an operational mitigation.

04 RECOMMENDATIONS

Beyond this audit

Four measures that extend the protection of this review after launch, independent of any individual finding.

R-01	Bug bounty	Stand up a public bounty on Immunefi or HackenProof with the top tier covering fund loss.
R-02	Invariant monitoring	Run an off-chain watcher that tracks <code>lastCheckpoint</code> age per <code>LeveragedToken</code> , flags pending redemptions stuck past the 1 hour cancel delay or executions returning false, and reconciles <code>totalAssets</code> against the EVM balance plus the HyperCore account. Page on divergence.
R-03	Checkpoint discipline	Keep a registry of every deployed <code>LeveragedToken</code> and have an agent checkpoint each one immediately before any <code>setStreamingFee</code> change, shrinking the retroactive window behind M-01. Regular sweeps also bound the stale-checkpoint exposure behind L-02.
R-04	Key hygiene	The <code>GlobalStorage</code> owner is the highest-value key in the system. It sets every fee parameter, swaps module addresses, and controls each token's agents. Keep it on a multisig with hardware signers.

05 CONTRACTS & ROLES

Core contracts

CONTRACT	ROLE
src/LeveragedToken.sol	ERC20 leveraged token minted and redeemed against USDC. Runs the streaming-fee checkpoint, the prepare-and-execute redemption queue, and agent-gated bridging between HyperEVM and HyperCore.
src/GlobalStorage.sol	Shared Ownable2Step parameter hub for every LeveragedToken, holding module addresses, the executor set, and the global streamingFee , redemptionFee , executeRedemptionFee , and minTransactionSize values.
src/HyperliquidHandler.sol	Stateless precompile reader reporting each token's HyperCore spot and perp USDC balances, the source of totalAssets pricing.

The review covered the eight pull requests listed in scope against commit 691b365, not the full codebase.

PRIVILEGED ROLES

Protocol owner. The Ownable2Step owner of GlobalStorage, honored by every LeveragedToken through globalStorage.owner() . Sets all fee parameters, swaps module addresses, pauses minting, and adds or removes each token's agents.

Agents. Up to three per token, registered as Hyperliquid API wallets. Call checkpoint , bridgeToCore , and bridgeToEvm to rebalance funds toward target leverage.

Executors. Addresses flagged in GlobalStorage.isExecutor . Process the queued redemptions through executeRedemptions .

06 FINDINGS

What we found

ID	TITLE	SEVERITY	STATUS
M-01	<code>streamingFee</code> updates apply retroactively to time elapsed before the change	MEDIUM	● ACKNOWLEDGED
M-02	Stale <code>lastCheckpoint</code> when <code>streamingFee</code> is set to zero allows retroactive fee charging	MEDIUM	● FIXED
L-01	Treasury fee share can round down to zero	LOW	● ACKNOWLEDGED
L-02	Stale <code>lastCheckpoint</code> on <code>mint()</code> charges retroactive fees	LOW	● ACKNOWLEDGED
L-03	Global parameters lock funds in depreciated <code>LeveragedTokens</code>	LOW	● ACKNOWLEDGED

● MEDIUM SEVERITY · 2 FINDINGS

M-01

MEDIUM

ACKNOWLEDGED

streamingFee updates apply retroactively to time elapsed before the change

LeveragedToken::_checkpoint reads streamingFee from GlobalStorage at the moment the checkpoint executes and applies that single rate to the entire period since the last checkpoint:

```
// LeveragedToken.sol - _checkpoint
uint256 timeElapsed_ = to_ - $.lastCheckpoint;
uint256 percentOfYear_ = (timeElapsed_ * 1e18) / 365 days;
uint256 streamingFee_ = $.globalStorage.streamingFee(); // current rate, no history
uint256 totalAssets_ = totalAssets();
uint256 annualFee_ = totalAssets_.mul(streamingFee_).mul($.targetLeverage);
uint256 periodFee_ = annualFee_.mul(percentOfYear_);
```

GlobalStorage::setStreamingFee overwrites the global value. It keeps no history, records no change timestamp, and forces no checkpoint on any LT. For every LT whose lastCheckpoint < block.timestamp when setStreamingFee is called, the next _checkpoint charges the new rate for the full timeElapsed_, including the portion that accrued under the old rate.

The owner cannot work around this by checkpointing every LT first: checkpoint() is onlyAgents, so the GlobalStorage owner is not authorized to call it, and no on-chain registry enumerates all LeveragedTokens, so even an agent cannot iterate them atomically in the same block as setStreamingFee.

With totalAssets = 1,000,000 USDC, targetLeverage = 3x, and a rate change from 2% to 10% at the midpoint of a 100-second idle period:

WINDOW	RATE	CORRECT FEE	CHARGED
t = 0 to 50s	2%	\$9.51	nothing yet
t = 50s to 100s	10%	\$47.56	nothing yet
Checkpoint at t = 100s	10% over 100s	\$57.08	\$95.13

IMPACT

Holders are overcharged by \$38.05 in the example, 67% above the correct integral, applied silently via `_payStreamingFee` at the next user interaction. The symmetric case also occurs: lowering the rate retroactively undercharges the period before the change, quietly crediting holders fees the protocol was entitled to under the old rate.

RECOMMENDATION

Store the rate-change timestamp and the previous rate in `GlobalStorage`, and split the fee integral in `_checkpoint` whenever the last checkpoint precedes the most recent rate change:

```
// GlobalStorage.sol
uint256 public previousStreamingFee;
uint256 public streamingFeeChangedAt;
function setStreamingFee(uint256 streamingFee_) external override onlyOwner {
    previousStreamingFee = streamingFee;
    streamingFeeChangedAt = block.timestamp;
    emit SetStreamingFee(streamingFee, streamingFee_);
    streamingFee = streamingFee_;
}
```

In `_checkpoint`, if `lastCheckpoint < streamingFeeChangedAt ≤ to_`, compute the fee as two segments, `[lastCheckpoint, streamingFeeChangedAt]` at `previousStreamingFee` and `[streamingFeeChangedAt, to_]` at the current `streamingFee`. This keeps the mitigation $O(1)$ per checkpoint as long as the owner makes at most one rate change per LT-idle period.

M-02

MEDIUM

FIXED

Stale `lastCheckpoint` when `streamingFee` is set to zero allows retroactive fee charging

In `LeveragedToken::_checkpoint`, when the calculated `periodFee_` is zero but `totalAssets_` is positive, the function returns early without updating `lastCheckpoint`:

```
// LeveragedToken.sol
uint256 annualFee_ = totalAssets_.mul(streamingFee_).mul($.targetLeverage);
uint256 periodFee_ = annualFee_.mul(percentOfYear_);
_payStreamingFee(periodFee_);
if (periodFee_ == 0 && totalAssets_ > 0) return; // @audit lastCheckpoint NOT updated
$.lastCheckpoint = to_;
```

This is an anti-rounding-loss mechanism: when `timeElapsed` is too small to produce a non-zero fee, the checkpoint stays frozen so micro-periods accumulate until a fee can be charged.

However, when the owner sets `streamingFee` to 0 via `GlobalStorage::setStreamingFee`, which is allowed since there is no `> 0` validation, `periodFee_` is always 0 regardless of how much time passes. The early return fires every time, and `lastCheckpoint` freezes indefinitely.

IMPACT

When `streamingFee` is later restored to a non-zero value, the next `_checkpoint` call computes `timeElapsed` from the stale `lastCheckpoint` and charges a retroactive streaming fee for the entire zero-fee period.

RECOMMENDATION

Update `lastCheckpoint` when `streamingFee` is zero, since there is no rounding remainder to accumulate:

```
_payStreamingFee(periodFee_);
-if (periodFee_ == 0 && totalAssets_ > 0) return;
+ if (periodFee_ == 0) {
+   if (totalAssets_ == 0 || streamingFee_ == 0) {
+     $.lastCheckpoint = to_;
+   }
+   return;
+ }
$.lastCheckpoint = to_;
```

This preserves the anti-rounding-loss behavior for short periods with `streamingFee > 0`, while correctly advancing the checkpoint when there is genuinely nothing to accumulate.

● LOW SEVERITY · 3 FINDINGS

L-01

LOW

ACKNOWLEDGED

Treasury fee share can round down to zero

`_checkpoint()` skips updating `lastCheckpoint` only when `periodFee_ == 0`. When `periodFee_ > 0`, it pays the fee and advances the checkpoint. In `_payFees()`, however, the treasury split rounds down:

```
uint256 treasuryAmount_ = feeAmount_.mul(treasuryFeeShare_);
```

IMPACT

For a small `feeAmount_`, `treasuryAmount_` rounds down to 0 and all fees go to the `feeHandler`. The checkpoint still advances, so the treasury's share of those periods is lost rather than accumulated. If rounding to zero is avoided in `_checkpoint()`, rounding to zero here should be avoided too.

RECOMMENDATION

Consider one of these:

1. Do not pay fees immediately. Accumulate them and pay once the amount is meaningful.
2. If `treasuryAmount_` rounds down to 0, skip updating the checkpoint.

L-02

LOW

ACKNOWLEDGED

Stale `lastCheckpoint` on `mint()` charges retroactive fees

`_checkpoint()` intentionally does not update `lastCheckpoint` when `periodFee_ = 0`, to avoid rounding loss:

```
if (periodFee_ = 0 && totalAssets_ > 0) return;  
$.lastCheckpoint = to_;
```

This is fine while `totalAssets_` stays flat or decreases, but not for minting, where `totalAssets_` increases. If someone mints after a long period of low `totalAssets_`, the next fee accrual uses the stale timestamp and charges fees over the entire elapsed time using the new, larger `totalAssets`.

IMPACT

The minter pays retroactive streaming fees for time their capital was never in the token, applied silently at the first accrual after the mint.

RECOMMENDATION

Consider either of these:

1. Always update `lastCheckpoint`, but calculate `periodFee_` at higher precision. Pay what you can and store the remainder to accumulate until it is large enough to pay.
2. Do not skip updating `lastCheckpoint` when a mint happens.

L-03

LOW

ACKNOWLEDGED

Global parameters lock funds in depreciated LeveragedTokens

Two global parameters in `GlobalStorage` create independent fund-locking vectors for depreciated `LeveragedTokens`.

Vector 1: the global minimum transaction size blocks entry into the redemption flow. Both

`LeveragedToken::redeem` and `LeveragedToken::prepareRedeem` enforce a minimum base amount check against the single global `minTransactionSize` :

```
// LeveragedToken.sol
uint256 baseAmount_ = ltToBaseAmount(ltAmount_);
if (baseAmount_ < $.globalStorage.minTransactionSize()) revert BelowMinTransactionSize();
```

When a leveraged token's `exchangeRate` drops significantly, for example a 3x long LT during a major price drawdown, the USDC value of a user's LT holdings falls below `minTransactionSize` , making redemption impossible, possibly even for users who hold the entire remaining supply.

Vector 2: the flat execute-redemption fee blocks execution of already-prepared redemptions. Even a user who passed the `minTransactionSize` check during `prepareRedeem` can be blocked, because

`_executeRedemption` recalculates `baseAmount_` at the current, potentially worse, exchange rate and subtracts a flat fee:

```
// LeveragedToken.sol - _executeRedemption
uint256 redemptionFee_ = _redemptionFee(baseAmount_, true);
if (baseAmount_ < redemptionFee_) return false;
```

`executeRedemptionFee` is a fixed USDC amount set globally. If the exchange rate drops between `prepareRedeem` and `executeRedemptions` , the recalculated `baseAmount_` can fall below `percentFee_ + executeRedemptionFee` , so `_executeRedemption` silently returns `false` . The user's tokens stay locked in `pendingRedemptions` until the `cancelRedeem` timeout.

IMPACT

The owner cannot lower `executeRedemptionFee` or `minTransactionSize` for a single struggling LT without affecting all healthy LTs, so holders of a depreciated token can be locked out of redemption entirely.

RECOMMENDATION

Allow both `minTransactionSize` and `executeRedemptionFee` to be overridden per `LeveragedToken`, falling back to the global defaults when no override is set. This lets the owner lower thresholds for depreciated LTs without weakening protections on healthy ones:

```
// In LeveragedTokenStorage:
uint256 minTransactionSizeOverride; // 0 = use global
uint256 executeRedemptionFeeOverride; // 0 = use global
```